FIFO Verification using SystemVerilog

Jinkyu Lee
jl9c23@soton.ac.uk

Abstract

This report presents the verification of a parameterized FIFO (First-In, First-Out) buffer implemented in SystemVerilog RTL. The main aim is to ensure that the design meets its specification and to analyse its functional correctness under various inputs. A structured SystemVerilog testbench that utilized constrained random stimulus was developed. In addition, assertions and coverage-driven verification techniques were used to verify corner cases such as simultaneous read/write operations and full/empty boundary conditions. Simulation results show that the testbench achieved approximately 98% functional coverage and 100% line coverage, and verified its functionalities based on most input combinations. However, additional verification is needed to cover some remaining corner cases.

1. Introduction

The design under verification is a parameterizable FIFO (First-In, First-Out) buffer, which stores input data in the order it was written and outputs data in the same order [1]. It can also execute write and read operations at the same time. It has several state flags such as full, almost_full, empty, and almost_empty, and its depth can be determined using parameter. The almost_full and almost_empty flags indicate threshold states for write and read operations, respectively.

Verifying the design through only simulation is not sufficient [2]. Verification is crucial to ensure functional correctness and reliable operations by checking various scenarios and corner cases such as simultaneous write/read and operation in full or empty states. Through this verification, errors that may occur under unexpected input conditions can be detected.

The main objective of verification is to ensure that the design meets its specification, testing as many possible input combinations as possible, and measuring verification metrics such as functional and code coverage. In addition, assertions were used to verify that the FIFO state flags behave logically.

To achieve these tasks, a component-based testbench environment was built, using a generator, a driver, a monitor, and a scoreboard. This modular testbench architecture improves reusability and scalability. Each component communicates through transaction-based mailboxes.

2. Background

2.1 Design Verification Overview

Design verification is one of the processes in chip design that ensures a design behaves correctly under various possible input conditions and checks whether a design meets its specification. Since it is difficult to verify all functionalities through a simulation only, systematic verification is necessary which can test various input combinations and edge cases. Particularly, a verification strategy that considers various scenarios is required to detect errors that may occur in unexpected situations.

2.2 FIFO Architecture

A FIFO(First-In, First-Out) buffer that widely used in digital systems, stores input data in order and outputs it in the same order it was written. It indicates its current storage state using flags such as full and empty, and represents its thresholds through almost_full and almost_empty flags. These flags ensure correct operation of the FIFO.

2.3 Verification Methodology

A typical SystemVerilog verification environment consists of several components such as a generator that generates random stimulus, a driver that drives this stimulus to the DUT, a monitor that observes

DUT outputs, and a scoreboard that compares DUT outputs against a reference model. This testbench architecture allows automated verification from various input generation to verifying DUT operations by communicating between components.

During verification, various input combinations can be generated by using constrained-random verification technique which allows to explore different behaviors of a design [3]. In addition, specific conditions or protocol can be verified automatically through SystemVerilog assertions [4]. These techniques can detect design errors at an early stage.

2.4 Coverage Metrics

Coverage measures how much the design has been tested and is used to evaluate verification completeness [5]. Functional coverage checks whether specific functions or states are executed, and code coverage measures the percentage of design code executed during verification. Code coverage can include metrics such as line coverage and branch coverage, which are measured automatically by simulators.

3. Method

A SystemVerilog verification environment was developed using class-based testbench components. This environment was designed by separating roles such as input stimulus generation, driving to the DUT, output observation, and checking final results, and assigning each role to a specific component. The main verification metrics were data integrity, FIFO ordering correctness, status flag, assertions pass/fail, and the functional coverage. These metrics were used to verify key functionalities of the FIFO.

The verification environment consists of generator, driver, monitor, scoreboard, and coverage components. The generator creates various input stimuli, the driver applies these stimuli to the DUT, the monitor observes the input and output of the DUT, and the scoreboard checks outputs against a reference model.

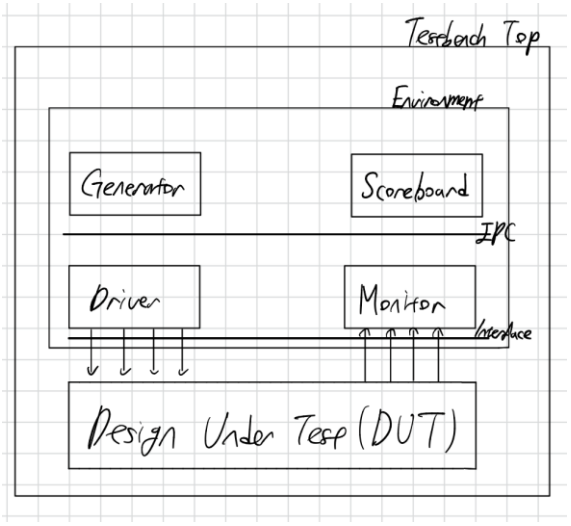

Figure 1: Testbench Architecture

The data transmission between each component was executed based on transaction class. Mailboxes were used for synchronized communication between components. Transactions generated from a generator were sent to the driver then applied to the DUT. The monitor captured DUT signals and sent as a transaction form to the scoreboard and coverage modules.

Diverse input stimuli were generated using the constrained-random technique. Constrained-random stimulus was used to test corner cases such as continuous read/write, simultaneous read/write operations, and boundary conditions between full and empty states. The write enable (we) and read enable (re)

signals were generated randomly except both signals become zero at the same time by defining a constraint in a transaction class.

FIFO functional verification was executed using the scoreboard and SystemVerilog assertions. In scoreboard, a queue was implemented as a reference model because it can be operated as FIFO buffer. Data integrity was checked by comparing input data against DUT output during the read operation. In addition, assertions were used to verify illegal and boundary conditions such as no write operation occurs when FIFO is full and correct behavior under full and empty conditions.

Coverage was collected using a covergroup such as write enable (we) and read enable (re), FIFO state flags, and data values. Functional coverage was used to check verification completeness, and all key functionalities were executed. Cross coverages were added to check the combination of we and re signals with FIFO states.

The verification process was designed to operate the generator, driver, monitor, scoreboard, and coverage parallel after the DUT reset. The generator task ended after creating a certain number of transactions, and it was synchronized with the scoreboard to proceed the next transaction. After processing all transactions, the simulation ended after a certain period of time.

4. Results

The verification environment processed all generated transactions without any runtime errors during simulation. The coverage results indicated that branch coverage was measured at 97.43%, and statement coverage achieved 100%. In addition, functional coverage measured based on covergroup reached 98.48%. Condition coverage and toggle coverage were achieved at 94.73% and 98.11%, respectively.

Assertion coverage was measured at 75%, which indicates that some assertion conditions were not activated during the simulation. The total coverage based on all instances was measured at 93.96%.

A total of 27 data matches and 281 mismatches were observed between the DUT output and the reference model during the read operation. Three assertions were applied, where AS2 and AS3 were satisfied without any violations throughout the simulation. However, AS1 showed 92 failures.

Metric	Value (%)
Functional	98.48
Statement	100.00
Branch	97.43
Condition	94.73
Toggle	98.11
Total	93.96

Table 1: Coverage Summary Table

```
# Data Matches:  27
# Data Mismatches: 281
# [GEN]: WE: 1, RE: 0, WRDATA:  2, RDDATA:  0
# [MON]: WE: 1, RE: 0, WRDATA:  2, RDDATA:  0
# [SCO]: FIFO is Full
# ** Error: Assertion error.
#    Time: 8115 ns Started: 8115 ns  Scope: tb.fif.AS1 File: tb_fifo.sv Line: 43
# ** Note: $finish    : tb_fifo.sv(416)
#    Time: 8215 ns  Iteration: 0  Region: /tb_fifo_sv_unit::environment #(4)::run
# 1
# Break in Task tb_fifo_sv_unit/environment::post_test at tb_fifo.sv line 416
VSIM 11> coverage report -summary
# Coverage Report Totals BY INSTANCES: Number of Instances 4
#
#     Enabled Coverage              Bins      Hits    Misses    Weight  Coverage
#     ----------------              ----      ----    ------    ------  --------
#     Assertions                       4         3         1         1    75.00%
#     Branches                        39        38         1         1    97.43%
#     Conditions                      19        18         1         1    94.73%
#     Covergroups                      1        na        na         1    98.48%
#         Coverpoints/Crosses         11        na        na         1       na
#             Covergroup Bins         59        58         1         1    98.30%
#     Statements                     153       153         0         1   100.00%
#     Toggles                        106       104         2         1    98.11%
# Total coverage (filtered view): 93.96%
#
#
```

Figure 2: Coverage Report Summary

5. Discussion

According to the simulation results, 100% statement coverage indicates that all lines of code were executed. This means that all operation paths such as reset, write, read, simultaneous write/read, full and empty state are executed. Functional coverage was measured approximately 98%, which indicates most input combinations and state conditions were verified. However, the reason why some cross coverages were not fully verified, certain rare combinations did not occur sufficiently due to limited number of applied stimuli.

In the scoreboard, the queue was implemented as a reference model and verified data integrity and ordering. Although high code coverage was achieved, the logs showed repeated scoreboard mismatches. This indicates that the DUT did not fully operate as the expected FIFO behavior and demonstrates that high coverage does not guarantee functional correctness.

Through waveform analysis, two main control issues were identified. First, the empty flag was asserted when count is 1 not 0. This results in enabling read operations when the FIFO is empty and causing count underflow. Second, simultaneous write/read operations did not maintain the correct count value.

The verification environment effectively verified DUT functionalities by detecting these errors. This represents that the testbench effectively identified design errors while achieving high coverage. Further development can be achieved by increasing the number of randomized transactions, adding corner case tests, and verifying with various parameter conditions to improve verification completeness.

6. Conclusion

In conclusion, a parameterized FIFO buffer implemented in SystemVerilog was verified in this report. A structured testbench that consisted of a generator, a driver, a monitor, a scoreboard, and a coverage module was developed. Constrained-random stimulus, assertions, and coverage-driven verification techniques were applied. The simulation results showed high functional and code coverage, which indicates that most FIFO operations and primary states were sufficiently verified. However, data mismatches and assertion failures were observed. This indicates that further investigation of both design and testbench may be required. Overall, the verification environment was effective in both verifying FIFO operation and collecting code and functional coverage. In future work, verification completeness can be improved further by applying directed tests, sophisticated assertions, and additional corner-case scenarios to verify corner cases and unhit covergroup bins.

7. References

[1] P. P. Chu, *FPGA Prototyping by SystemVerilog Examples*. Hoboken, NJ, USA: John Wiley & Sons, 2018.

[2] J. Bergeron, *Writing Testbenches Using SystemVerilog*. New York, NY, USA: Springer Science & Business Media, 2007.

[3] C. Spear, *SystemVerilog for Verification*. New York, NY, USA: Springer Science & Business Media, 2006.

[4] E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, *SVA: The Power of Assertions in SystemVerilog*. New York, NY, USA: Springer, 2014.

[5] B. Wile, J. Goss, and W. Roesner, *Comprehensive Functional Verification*. Burlington, MA, USA: Elsevier, 2005.